

Lab 1 – 7-Segment Display Driver

Table of Contents

1. Introduction	1
1.1 Laboratory Logbook.....	2
2. Part A – 7-Segment Decoder Implementation.....	2
2.1 Introduction.....	3
2.2 Implementation.....	3
• Finite State Machine (FSM).....	3
• Modules	3
• Always blocks.....	3
2.3 Verification.....	3
2.4 Experiment / Final Implementation	4
3. Part B – Four-Digit Display Driver	4
3.1 Introduction.....	5
3.2 Implementation.....	5
3.2.1 Sync_2ff	6
• State Machine (FSM).....	6
• Modules	6
• Always blocks.....	6
3.2.2 Debouncer.....	6
• State Machine (FSM).....	7
• Modules	8
• Always blocks.....	8
3.2.3 Counter.....	10
• State Machine (FSM).....	11
• Modules	11
• Always blocks.....	11
3.2.4 Anode driver	11
• Finite State Machine (FSM).....	12
• Modules	12

• Always blocks.....	13
3.2.5 FourDigitLEDDriver	13
3.3 Verification.....	13
3.4 Experiment / Final Implementation	16
4. Part C – Stepwise Message Rotation using Button	16
4.1 Introduction.....	17
4.2 Implementation.....	17
4.3 Verification.....	18
4.4 Experiment / Final Implementation	19
5. Part D – Stepwise Message Rotation with Fixed Delay	21
5.1 Introduction.....	21
5.2 Implementation.....	21
5.3 Verification.....	22
5.4 Experiment / Final Implementation	23

Name: Ioannis Papadopoulos, Maria Ilektra Tzormpatzaki

Student ID: 03899, 03993

Date: 11/7/2025

Board: Boolean Board (Spartan-7 FPGA)

1. Introduction

The objective of this laboratory exercise is to design and implement a 7-segment display driver controlling four LED digits on the FPGA board. The system cyclically displays a 16-character message. The work is divided into four parts:

- The 7-segment decoder
- The four-digit driver
- The button-controlled message rotation
- The timed rotation implementation.

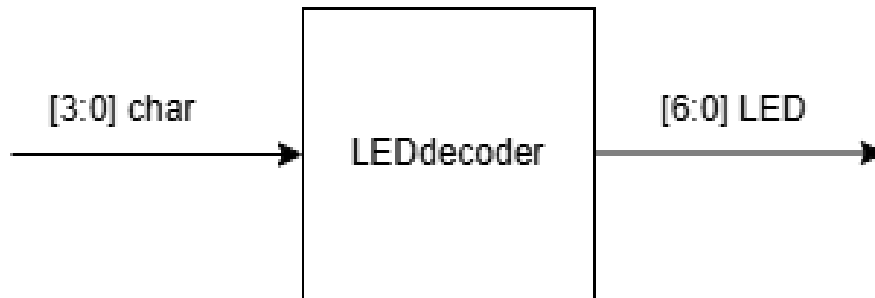
1.1 Laboratory Logbook

Throughout the laboratory sessions, a detailed logbook was maintained to document the design process, the issues encountered, and the corresponding solutions. Each entry recorded the date, the tasks completed, simulation or synthesis results, and any observations related to circuit behavior on the FPGA board.

The purpose of this logbook is to track progress systematically and to provide insight into the reasoning behind design decisions. It also serves as a valuable troubleshooting reference, especially when unexpected results occur during testing or hardware implementation.

2. Part A – 7-Segment Decoder Implementation

2.1 Introduction



The LEDdecoder module is a combinational circuit that converts a 4-bit input value into the corresponding 7-bit output required to display hexadecimal digits on a seven-segment LED. The module has one input port “char[3:0]” and one output port “LED[6:0]”. Each output bit corresponds to one of the display cathodes (A–G), using active-low logic, meaning that a segment turns on when its signal is 0. The circuit does not use any clock or sequential logic elements. It produces the correct display pattern directly from the input value.

2.2 Implementation

- **Finite State Machine (FSM)**

No finite state machine is used in this part of the design. The decoder is purely combinational and does not require states or transitions, since the output depends only on the current input value.

- **Modules**

The design consists of a single module, named LEDdecoder. This module serves as a reusable building block for larger designs, such as the multi-digit display driver used in later parts of the lab.

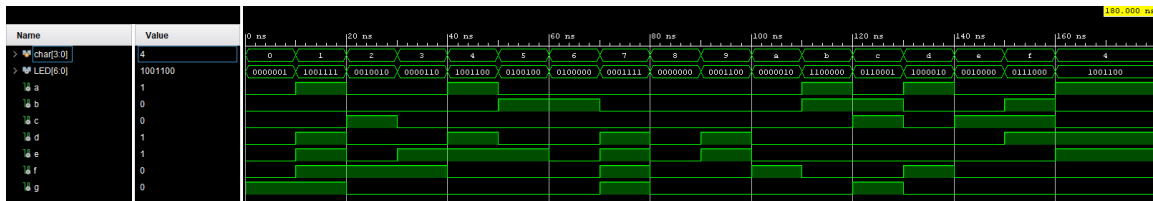
- **Always blocks**

The entire behavior is contained in one always block that is sensitive to the input signal char. Whenever char changes, the block executes and updates the output LED vector. Inside this block, a case structure is used to define the mapping between each hexadecimal input value (0 to F) and the corresponding 7-bit pattern for the segments.

2.3 Verification

The verification of the LEDdecoder module was carried out through a behavioral simulation using a dedicated testbench. The testbench instantiates the decoder and systematically applies all 16 possible 4-bit input combinations, ranging from 0000 to 1111. This process was implemented using a for-loop, allowing each input value to be applied for a short time interval (10ns) before moving to the next one. During this loop, the corresponding 7-bit LED output was monitored and compared against the expected hexadecimal segment patterns.

After completing the loop, the input signal was intentionally kept constant for several consecutive cycles. This step verified that the output of the decoder remained stable and did not fluctuate when the input stayed unchanged.



The simulation confirmed that every input produced the correct LED pattern, consistent with the active-low configuration of the Boolean Board. The output responded immediately to input changes and stayed steady when the input was held constant. No unexpected glitches or timing issues were observed, confirming that the LEDdecoder operates correctly and reliably as a purely combinational circuit.

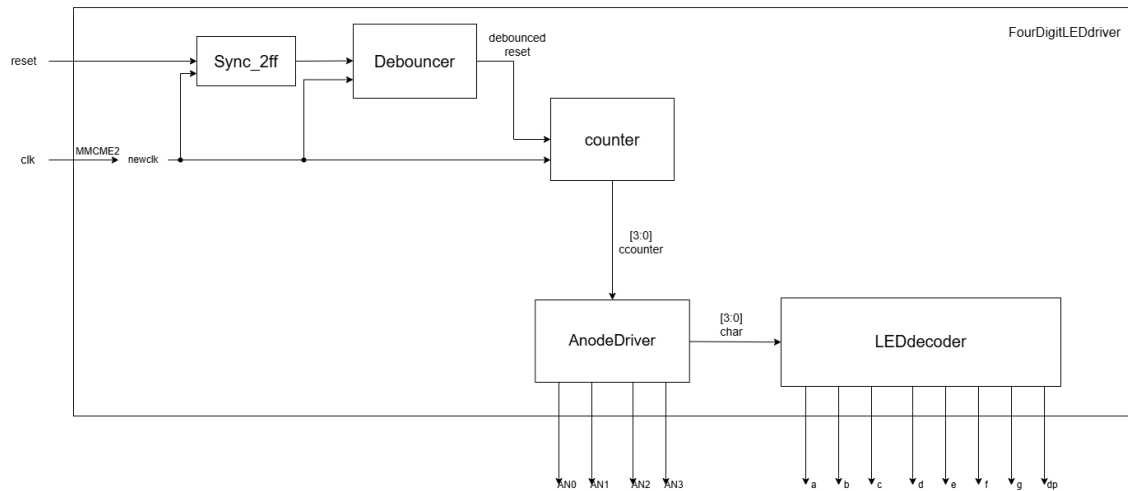
2.4 Experiment / Final Implementation

Part A could not be directly tested on the Boolean Board because it only involved the LEDdecoder module. This module by itself does not include timing, control, or hardware interfacing logic, making it unsuitable for standalone testing on the FPGA.

3. Part B – Four-Digit Display Driver

3.1 Introduction

The purpose of Part B is to extend the functionality of the single-digit 7-segment decoder developed in Part A into a fully operational four-digit display driver. While the previous part focused on generating the correct segment pattern for a single hexadecimal input, this stage introduces the timing, synchronization, and control logic required to drive multiple digits.

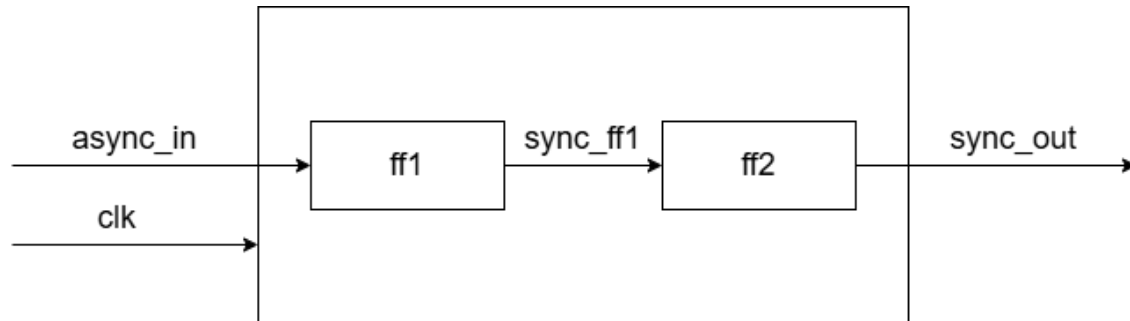


In this part, the system must sequentially activate each of the four displays at a high refresh rate, ensuring that only one digit is illuminated at any given moment. The rapid switching between digits creates the illusion of all four being continuously lit. To achieve this, the circuit incorporates clock division and counter-based control.

3.2 Implementation

3.2.1 Sync_2ff

This module ensures proper synchronization of asynchronous signals with the main system clock. It passes the external signal (in this case, the reset pushbutton) through two cascaded D flip-flops triggered on the rising edge of the clock. This prevents metastability and ensures that the reset signal is cleanly aligned with the clock domain of the FPGA logic.



- **State Machine (FSM)**

This module does not contain any FSM, as there are no states or transitions to control. The behavior is purely sequential: on every rising clock edge, the two flip-flops update their values according to the input.

- **Modules**

The circuit is composed of a single module (Sync_2ff) that includes two registers functioning as D flip-flops. This module can be easily reused in other parts of the design wherever asynchronous-to-synchronous signal conversion is required.

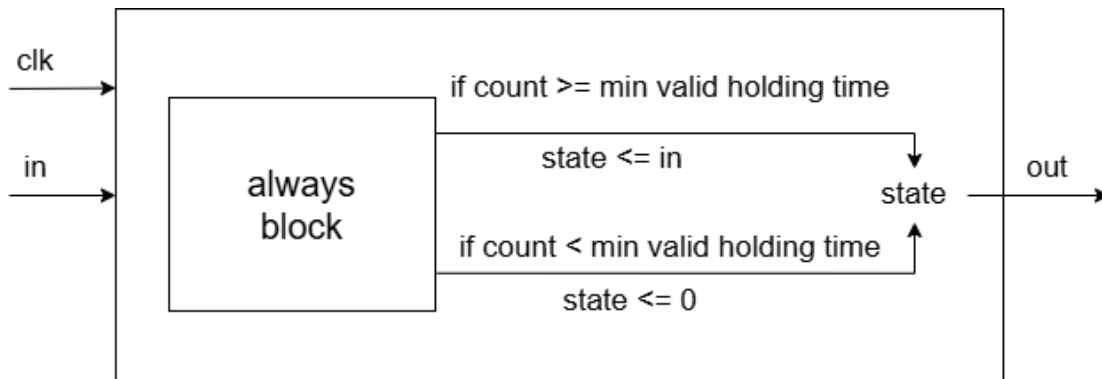
- **Always blocks**

The module contains a single always block triggered by posedge clk. Inside this block, the assignments define the two-stage synchronization chain:

- The first assignment stores the asynchronous input into sync_ff1.
- The second assignment transfers the value of sync_ff1 to the output sync_out on the next clock edge.

3.2.2 Debouncer

The Debouncer module is designed to filter out the mechanical noise and rapid signal fluctuations (“bouncing”) that occur when a physical pushbutton is pressed or released. Its purpose is to produce a clean, stable output signal (btn_out) that changes state only when the input (btn_in) has remained consistently high for a fixed amount of time in this case, 5 milliseconds.



- **State Machine (FSM)**

The Debouncer can be modeled as a simple FSM with two main states, defined by the internal register state:

- State = 0: Button is considered released.
- State = 1: Button is considered pressed.

The FSM changes between these two states only after the input (btn_in) remains stable for a defined period for example 5 ms in this design.

To achieve this delay, a counter is used. Each time btn_in differs from state, the counter starts incrementing at every rising edge of the clock. Once the counter reaches the constant value COUNT_MAX = 25,000, the FSM updates its state to match the new button value.

This value is derived from the desired debounce time and the clock frequency:

$$COUNT_MAX = 5 \text{ ms} / (0.2 \text{ } \mu\text{s}) = 0.005 / 0.0000002 = 25,000$$

At a 5 MHz clock frequency (period = 0.2 μ s), it takes 25,000 clock cycles to span 5 ms, ensuring the input must remain stable for that entire time before being accepted as valid.

- **Modules**

The Debouncer module operates as a single functional block within the system. As inputs it receives the clock and the synchronized button signal (btn_in) from Sync_2ff. As an output is the debounced version (btn_out).

- **Always blocks**

1. Clocked always block (posedge clk):

This block implements the main FSM logic and the counter mechanism. When the input signal changes (btn_in $\neq$ state), the counter increments until it reaches COUNT_MAX, at which point the state is updated to the new input value and the counter resets.

```
reg state = 1'b0;

always @(posedge clk) begin
    if (btn_in != state) begin
        count <= count + 1;
        if (count >= COUNT_MAX-1) begin
            state <= btn_in;
            count <= 0;
        end
    end else begin
        count <= 0;
        state <= 0;
    end
end
```

The else branch handles the condition where the input matches the state (meaning the button is zero again). It resets the counter and returns the state to zero, ensuring that the output pulse lasts for only one clock cycle.

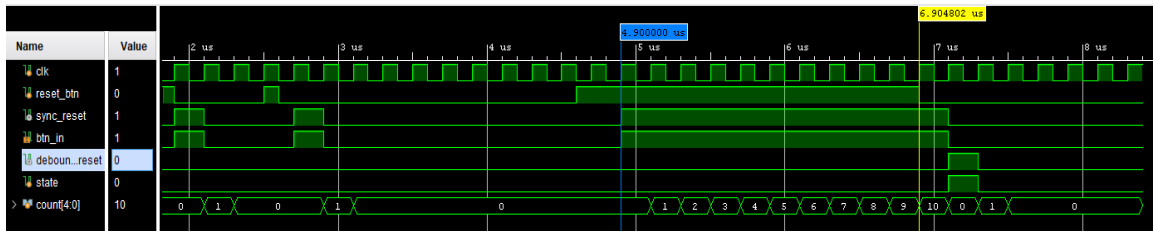
2. Combinational always block (on state):

The second always block simply assigns the output to reflect the current stable state:

```
always @(state) begin
    btn_out = state;
end
```

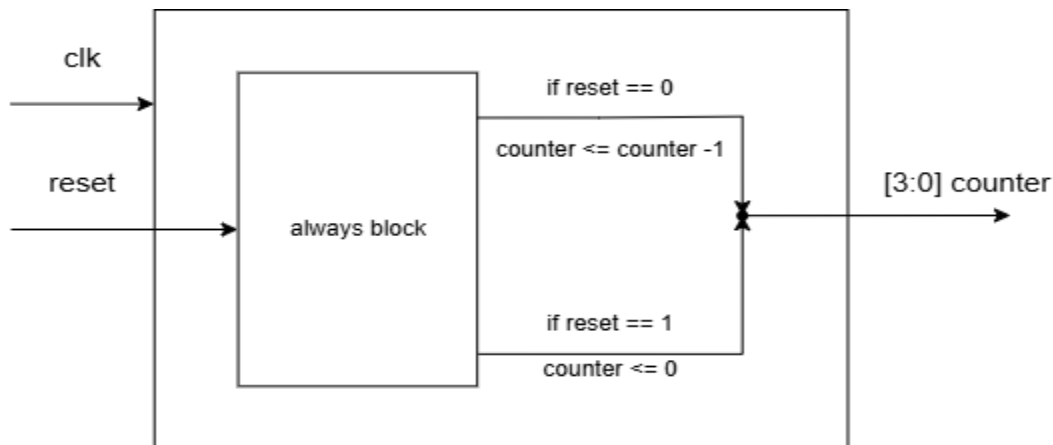
This ensures that btn_out is updated whenever state changes.

After testing the module with $COUNT_MAX = 10$ so that if the `btn_in` stays for 2us (for simulation purposes) it becomes a valid signal, we got the following waveform:



After exactly 2us of the `btn_in` being pressed, the `btn_out` rises to 1 and stays there for one period, so effectively the circuit would act like the reset button was pressed once. Note that the simulation also included the `Sync_2ff` module, that's why the `btn_in` rises 2 posedges of a clock after the reset signal went high.

3.2.3 Counter



The Counter module is a simple sequential circuit that generates a continuously decreasing 4-bit value used for timing and control within the display driver system.

- **State Machine (FSM)**

This module does not explicitly use a finite state machine. However, its behavior can be interpreted as a simple state machine with 16 states (from 0000 to 1111), where each clock pulse transitions to the next state in a predefined sequence. The transitions occur in reverse binary order due to the subtraction operation.

- **Modules**

The design consists of a single module (counter) containing one 4-bit register (counter) that stores the current count. The inputs of this module are the clock and the reset signal. Whereas, the output of this module is the current value of this register, which can be used by other modules (e.g., the anode driver) as a timing reference.

- **Always blocks**

The module includes one always block triggered by the positive edges of both the clock and the reset signals. The logic inside this block defines two behaviors:

- When reset is high, the counter immediately resets to zero (4'b0000), ensuring a known starting state.
- When reset is low, the counter decrements its value by one at every rising edge of the clock.

3.2.4 Anode driver

The Anode_driver module is responsible for controlling which of the four 7-segment display digits (anodes) is active at a given moment and for selecting the corresponding 4-bit character to be displayed on that digit. It acts as the interface between the counter and the LEDdecoder, ensuring correct timing and synchronization during the multiplexing process.

This module is a combinational circuit that derives its outputs directly from the value of the input counter. It determines which anode should be active and which character should be sent to the LEDdecoder. The circuit operates continuously, updating its outputs whenever the counter changes value.

- **Finite State Machine (FSM)**

The Anode_driver module functions as a (FSM), where each counter value represents a distinct state. Each state defines which digit is illuminated and what character is displayed on that digit. The FSM transitions sequentially through 16 states, controlled by the counter input. The following state table is displaying the sequence 0123.

Counter Value	an3	an2	an1	an0	char
0000	1	1	1	1	0
0001	1	1	1	1	3
0010	1	1	1	0	3
0011	1	1	1	1	3
0100	1	1	1	1	3
0101	1	1	1	1	2
0110	1	1	0	1	2
0111	1	1	1	1	2
1000	1	1	1	1	2
1001	1	1	1	1	1
1010	1	0	1	1	1
1011	1	1	1	1	1
1100	1	1	1	1	1
1101	1	1	1	1	0
1110	0	1	1	1	0
1111	1	1	1	1	0

The FSM is made so that each anode is activated once per full counter cycle, while its associated character data is properly prepared 2 states in advance and held for 1 state afterwards. The pattern allows only one anode to be active at any time (logic 0), preventing unwanted overlapping illumination of digits (ghosting).

- **Modules**

The `Anode_driver` is implemented as a single module. It receives the 4-bit counter input and produces five outputs: the four anode control signals (`an3`, `an2`, `an1`, `an0`) and the 4-bit char output, which is fed into the `LEDdecoder`. The module does not contain submodules. Instead, it directly implements the control logic for digit selection.

- **Always blocks**

The module includes a single `always` block sensitive to changes in the input counter. Inside this block, a case statement specifies the output values for each counter state. Each case assigns the corresponding anode pattern and character value. This structure ensures that the outputs are updated immediately whenever the counter changes, maintaining real-time synchronization with the multiplexing sequence.

3.2.5 FourDigitLEDdriver

The `FourDigitLEDdriver` module is the top-level integration unit of the four-digit 7-segment display system. It connects all the submodules developed in Part B including the Counter, `Anode_driver`, `LEDdecoder`, `Sync_2ff`, and `Debouncer` and manages the clock division through an MMCM (Mixed-Mode Clock Manager) primitive.

The top module performs three key functions:

1. **Clock Management:** Generates a lower-frequency clock suitable for driving the display without visible flickering, by using the `MMCME2`.

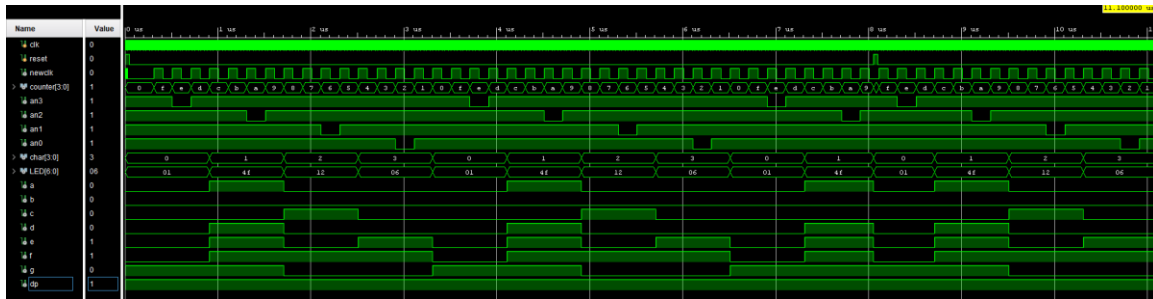
The `MMCM2_BASE` primitive is a built-in FPGA block that allows dynamic manipulation of the input clock signal. In this design, it is configured to multiply the base clock by 6 and then divide it by 120, resulting in a new clock (`newclk`) of approximately 5 MHz, corresponding to a 0.2 μ s period. This slower clock is used for display multiplexing and ensures that the digits switch fast enough for persistence of vision, while remaining slow enough for clean signal transitions.

The multiplication and division values were carefully chosen to comply with MMCM operating constraints. Not all integer ratios are supported due to hardware limits on the internal frequency ranges and phase alignment. The chosen configuration provides a stable output frequency and duty cycle suitable for display control.

2. **Manage the reset signal:** Ensures the external reset button input is synchronized using the “`Sync 2ff`” module and then pass it to the “`Debouncer`” module.
3. **Display Control Integration:** Coordinates digit multiplexing, character decoding, and LED output activation through the submodules.

3.3 Verification

The purpose of the verification phase for the FourDigitLEDdriver is to confirm that the fully integrated design behaves exactly as expected after synthesis and implementation. The post-implementation waveform analysis allows us to observe the real timing and signal interactions between all modules under the influence of the generated clock and reset signals.

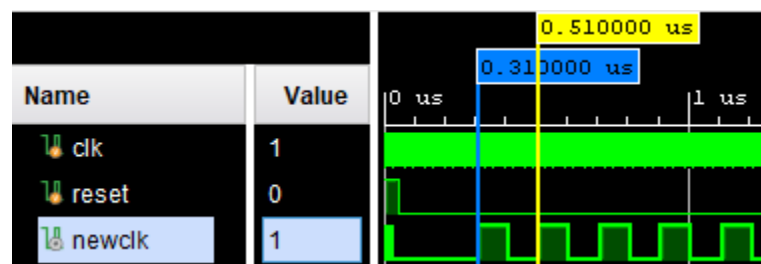


For the purposes of this waveform verification, the reset signal will not be passed through the Sync_2ff and Debouncer modules. These components were already validated separately in previous tests to confirm their correct operation. In this stage, the reset is applied directly to the circuit to provide a clean and immediate reset responses

The primary objectives of this verification are as follows:

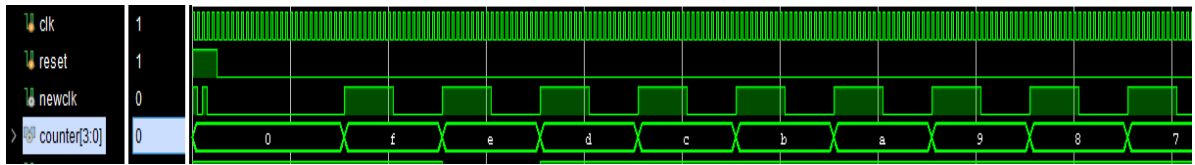
1. Clock Generation

Verify that the MMCM correctly produces the divided clock signal newclk with a period of approximately $0.2 \mu\text{s}$ (5 MHz).



The screenshot shows that 2 posedges of the newclk have a $0.2 \mu\text{s}$ distance, therefore the newclk is correctly generated.

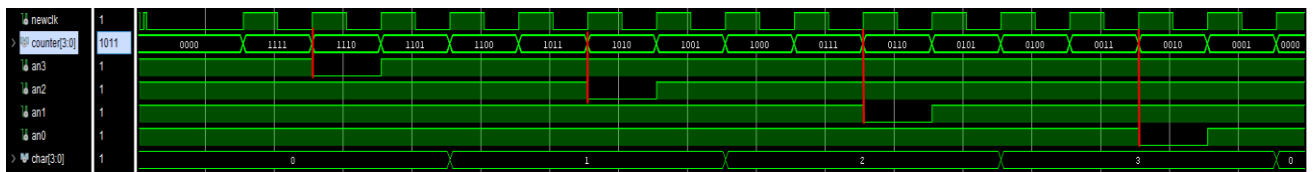
2. Counter Behavior:



Observe that the counter signal changes value on every positive edge of newclk, cycling through all 16 states as expected. This confirms the correct sequencing and timing behavior of the finite state machine implemented within the counter.

It appears that it's working as intended. The values are on the hexadecimal, but it's the correct ones.

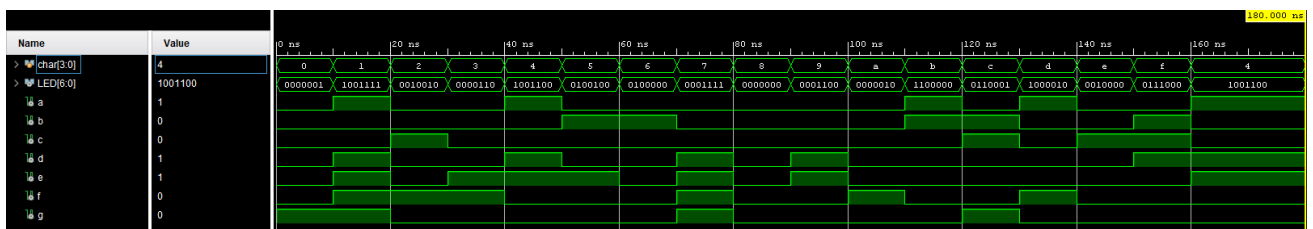
3. Anode and Character Synchronization:



Verify that the outputs of the Anode_driver module (an3–an0 and char) follow the correct pattern based on the counter's state.

The waveform shows each anode being activated in turn, with its corresponding character value prepared two clock cycles before the anode becomes active and remaining stable for one clock cycle after the anode is deactivated.

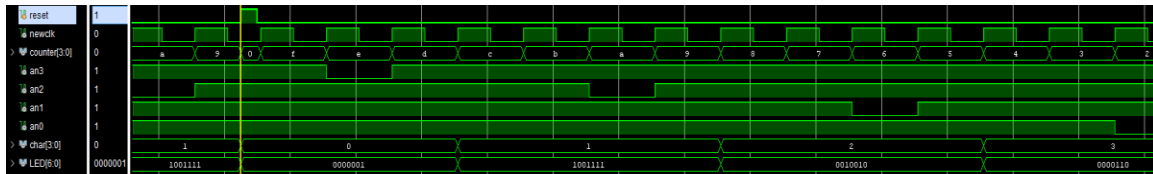
4. LED Output Validation:



Confirm that the LEDdecoder correctly translates each character value into the corresponding 7-bit LED output.

For each combination of char, the LED bus matches the expected segment pattern. Also the LED bus is correctly split in the cathodes.

5. Mid-Simulation Reset Test:



During the middle of the simulation, a second reset pulse will be applied to ensure that all modules correctly reinitialize to their default states, clearing all internal registers and outputs.

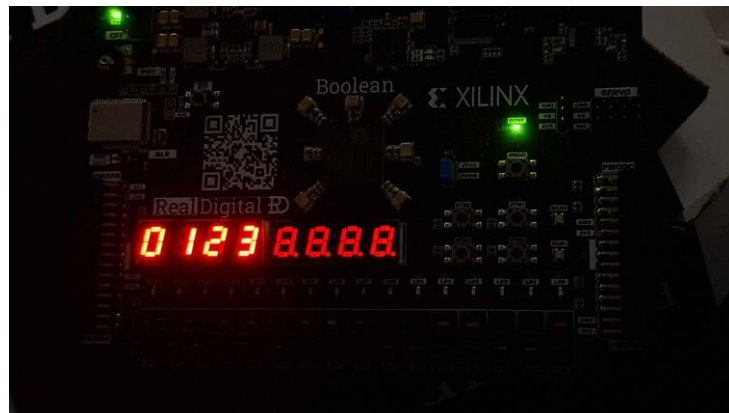
The waveform shows the counter resetting to zero and as a result, the anodes turned off and then the an3 lit up first. Also the LED output returning to an inactive state, after which normal operation resumes cleanly.

3.4 Experiment / Final Implementation

The programming of the Boolean Board was completed without errors. During the synthesis and implementation stages, Vivado produced a few warnings related to the MMCM configuration, but these were expected. After consulting with the instructors, the warnings were confirmed not to be a problem.

In the first testing, a ghosting effect was observed on digits 1 and 7. This issue was resolved in the following week, most likely due to improved setup timing of the char signal before each anode activation, ensuring that the correct value was stable prior to the digit lighting up.

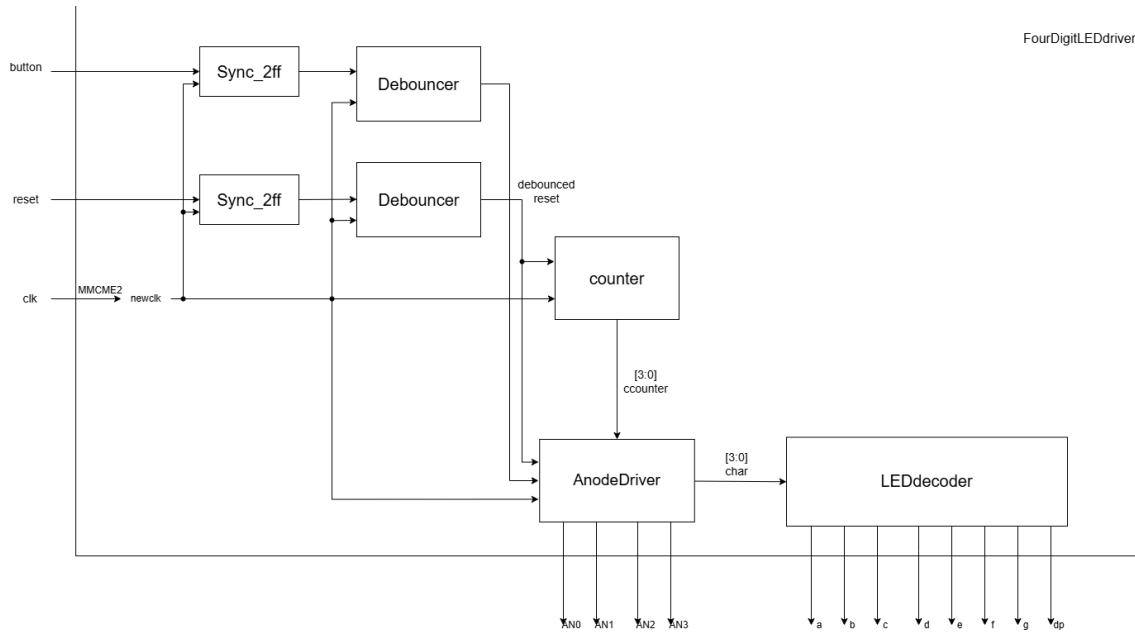
Aside from that, no other major problems were encountered



4. Part C – Stepwise Message Rotation using Button

4.1 Introduction

The purpose of Part C is to extend the functionality of the four-digit display system by introducing stepwise message rotation control through a pushbutton input. Unlike the continuous display behavior in the previous part, where the four digits showed fixed values in a repeating pattern, this stage allows the user to manually advance the displayed message one step at a time with each button press.



As shown in the Data flow the new button signal is being routed through the Sync_2ff and the Debouncer module, just like the reset button was before. The FourDigitLEDdriver is responsible for this routing.

4.2 Implementation

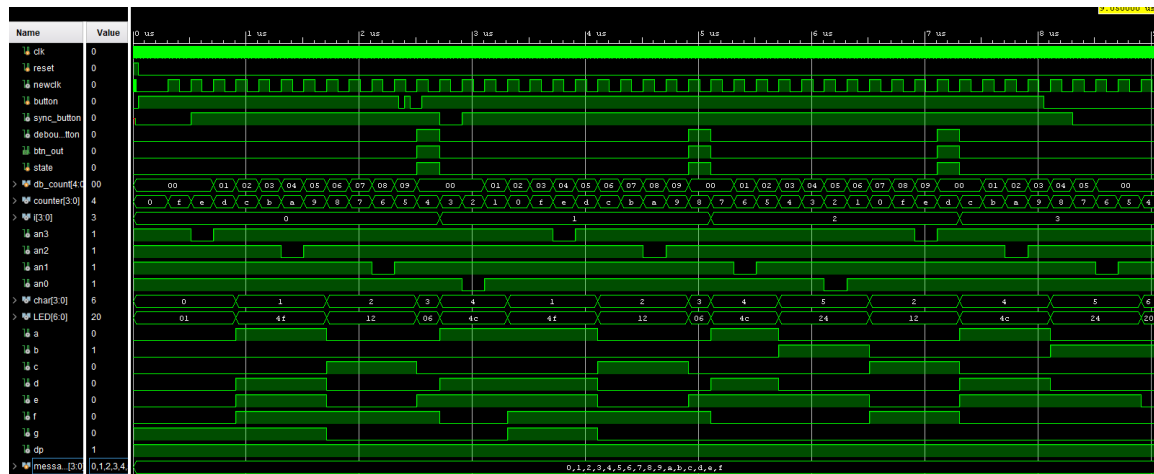
In this final part, the design architecture remains fundamentally the same as in Part B, the main modification occurs within the **Anode_driver** module. The updated **Anode_driver** module now includes the clock (**clk**) and reset (**reset**) as inputs, in addition to a new signal input called **button**, which represents the clean, debounced output of the user button. Inside this module, a register array named **message** is introduced to hold a sequence of hexadecimal values (from 0 to F). These values represent the message that will be displayed across the four digits.

When the circuit is reset (**reset** = 1), the message array is initialized with all hexadecimal values (0–15), and the index register **i** is reset to zero. This ensures that the display always starts from the 0123 sequence as Part B.

The stepwise rotation mechanism is implemented through the following logic: on every rising edge of the clock, if the button is pressed, the index register i increments by 1. This increment causes the anode driver to shift which part of the message array is being displayed. Consequently, each valid button press advances the displayed pattern by one position, effectively rotating the message on the display one step at a time.

4.3 Verification

The verification of Part C focuses on confirming that the new interactive button operates correctly and that all previously implemented modules continue to behave as expected under the updated design.



Note that the COUNTER_MAX of the Debouncer is only 10 and that the reset once again is not passed through a debouncer, for the simulation.

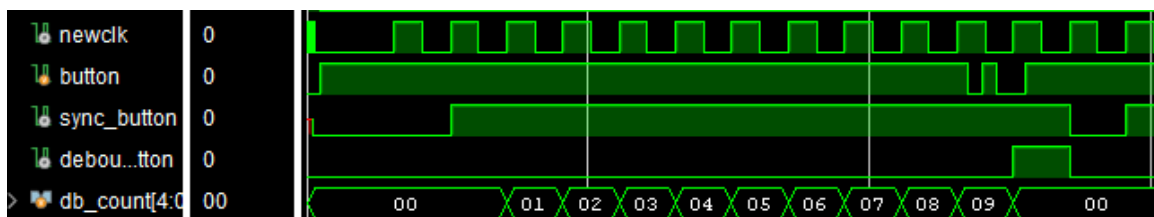
The primary objectives of this verification are as follows:

Message Initialization:

The Message and the index i can be seen in the waveform above initialized. Therefore, the circuit can be tested.

Button Synchronization and Debouncing:

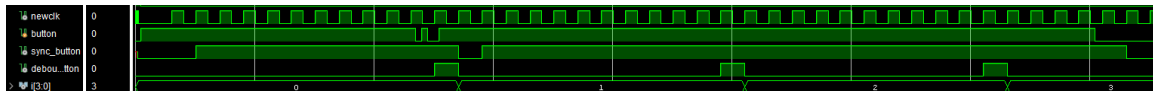
The button input is first passed through the Sync_2ff module to align it with the system clock and then through the Debouncer module.



The verification waveform shows that this works correctly. In the waveform, the “button” is the raw initial signal, the “sync_button” is the output of the Sync_2ff function which correctly rises 2 posedges after the initial signal. The “debounce button” is the output of the debounce that correctly rises after the “db_counter” reaches 10, meaning that the appropriate time of validation was surpassed.

Index Increment Behavior:

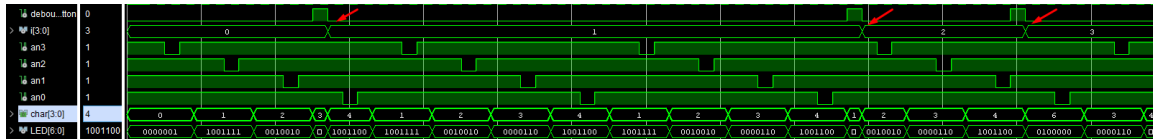
Each time the debounced button signal (debounce_button) goes high, the internal index register i within the Anode_driver must increment by exactly one on the next rising edge of the clock.



The waveform shows a clear, stepwise increase in the value of i corresponding to each valid button press, without multiple increments caused by bouncing.

Character Update Verification:

For every new value of the index i, the Anode_driver should output the corresponding set of character values from the message array. Also the LED should continue to correctly translate each char value into the 7-bit LED output pattern.



After adjusting the simulation so that the i doesn't change before a every single counter loop, we see that the char updates correctly, and so does the LED

4.4 Experiment / Final Implementation

The FPGA implementation for Part C completed successfully, with no synthesis or implementation errors reported by Vivado.

However, during the first hardware test, we noticed that the button was seemingly skipping steps in the sequence, each press would sometimes advance the message by more than one position. To investigate this, we reviewed the post-implementation waveforms at home, but everything appeared normal in simulation: the index i incremented exactly once per button press, and all signals behaved as expected.

When we tested again during the final lab session, the same behavior persisted. After further debugging, we discovered that the issue was not with the synchronization or logic, but rather with the Debouncer's sensitivity. The debounce duration had been set to only 5 ms, which proved too short for reliable operation with the physical pushbutton used on the FPGA board. Increasing the debounce interval by changing the COUNT_MAX value from 25,000 to 400,000 (corresponding to an 80 ms delay at 5 MHz) completely solved the skipping problem. After this change, the button produced stable, single-step rotations without unintended multiple increments.

However, a new issue appeared: when the button was held down for multiple seconds, the message sequence would continue advancing rapidly instead of remaining fixed after one step. Later, the instructors clarified that the intended behavior was for the system to advance only one step per button press, even if the button remained pressed.

With this clarification, we attempted to modify the Debouncer's logic to hold the output high for the entire duration of a long press rather than producing a single pulse. Our first idea was to remove the line inside the else branch that resets the state to zero:

```
end else begin
    count <= 0;
    state <= 0; // this line was forcing the pulse to last only one clock cycle
end
```

We realized that this line was the reason why the btn_out signal always lasted for a single pulse once the count reached the debounce threshold, state was set to 1 momentarily and then immediately reset to 0 on the next cycle. This caused the output to oscillate rapidly when the button was held down.

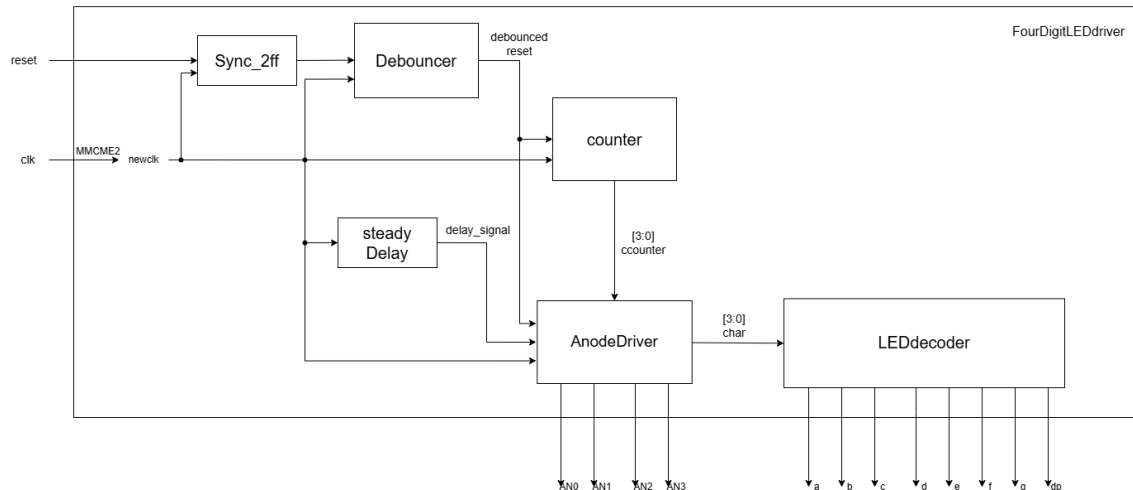
Unfortunately, our attempts to fix the logic by removing this line and adding additional conditional checks introduced new problems. The button started skipping multiple steps again, and in some cases, even a short press caused several rapid state changes. The implementation became unstable, and we were unable to fully restore its intended behavior before the lab session ended.

Ultimately, we reverted to the previous version of the code, which produced reliable stepwise rotation for single presses but still advanced continuously when the button was held down. Despite this minor limitation, the system was otherwise fully functional, the button correctly triggered message shifts, the display updated synchronously, and no other timing or electrical issues were observed.

5. Part D – Stepwise Message Rotation with Fixed Delay

5.1 Introduction

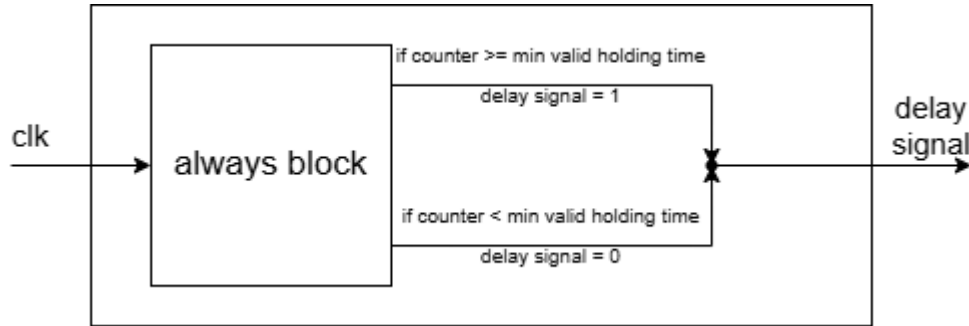
In Part D, the goal to replace the manual button control with an automatic, fixed-time rotation mechanism. Instead of relying on user input to trigger each message shift, the display now advances periodically based on a controlled timing signal generated within the FPGA. This modification transforms the system from an event-driven (button-based) design into a time-driven one, where message rotation occurs automatically at a constant rate.



As shown in the Data flow the new delay signal is being inserted into the directly to Anode Driver. The FourDigitLEDdriver is responsible for this routing.

5.2 Implementation

To achieve the periodic message rotation, a new module named `steady_delay` was introduced. This module generates a single-cycle pulse “`delay_signal`” at fixed time intervals determined by an internal counter.



The generated pulse is used to increment the message index “`i`” inside the `Anode_driver`, effectively shifting the displayed message at a consistent, hardware-defined rate.

The `steady_delay` module is a synchronous sequential circuit driven entirely by the clock. It contains a single counter that continuously increments on every rising edge of the clock. Once the counter reaches a predefined maximum value “`COUNT_MAX`”, it resets to zero and asserts the `delay_signal` output high for exactly one clock cycle.

The key timing parameter of the design is the “`COUNT_MAX`” value, which determines how long it takes before a new pulse is generated. In this implementation:

$$COUNTMAX = 8,388,607$$

Given the system clock frequency of 5 MHz (period = 0.2 μ s), this count value produces a pulse approximately every:

$$8,388,607 \times 0.2\mu s = 1.6777214 \text{ seconds}$$

This means that every ~1.68 seconds, the `delay_signal` goes high for one clock cycle, triggering the message rotation logic inside the `Anode_driver`.

5.3 Verification

The verification of Part D focuses on confirming that the newly added “`steady_delay`” module correctly generates periodic pulses at the intended intervals, and that these pulses successfully drive the automatic message rotation in the display system. Since the rest of the architecture remains unchanged from Part C, the main focus of this verification is the timing and synchronization of the delay signal and its interaction with the `Anode_driver`.



NOTE that the COUNTER_MAX of the “steady_delay” module is only 20 and that the reset once again is not passed through a debouncer, for the simulation.

The primary verification objectives are as follows:

1. Timing of the Delay Signal (delay_signal):

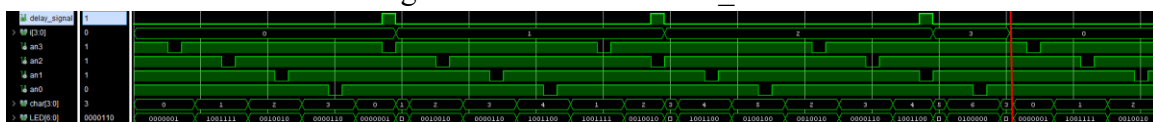
The main goal is to verify that the steady_delay module produces a single-cycle high pulse at the expected time intervals. With this configuration, the delay_signal should go high once every 20 clock cycles.



The waveform shows a clean, periodic pulse that stays high for exactly one clock cycle and then returns to zero, as intended.

2. Index Increment Behavior (i):

The next objective is to confirm that each rising edge of delay_signal correctly increments the index register i within the Anode_driver.



The waveform demonstrates that i increases by one immediately after each pulse, in synchronization with the clock, and that no extra increments occur between pulses. Also as in Part C, the corresponding char and LED outputs continue to function correctly and follow the updated index values.

NOTE that the red line in the waveform is to mark the point where a reset signal occurs, the values were correctly reset.

5.4 Experiment / Final Implementation

The implementation of Part D was completed successfully, and the entire design worked as intended without any major issues in Vivado

The only minor issue encountered was during the first test, where the COUNT_MAX parameter in the steady_delay module was accidentally left at the smaller simulation value (20) instead of being restored to 8,388,607. As a result, the message sequence was refreshing extremely fast instead of every 1.6 seconds as intended. After correcting this parameter, the display operated as intended.

Overall, Part D functioned correctly from the beginning once the timing parameter was set properly, demonstrating smooth and consistent automatic message rotation without any further adjustments or hardware-related problems.